Playwright & Node.js Interview Questions

Complete Study Guide — 37 Topics

Contents

01.1 — npm Installation

Topic group: Node.js & npm Ecosystem | **Sequence:** 1 of 37

Q1. What is npm?

npm (Node Package Manager) is the default package manager for Node.js. It installs, manages, and shares JavaScript/TypeScript packages and dependencies.

Q2. What command scaffolds a new Playwright project?

```
npm init playwright@latest
```

Creates config, example tests, and installs all required dependencies automatically.

Q3. What does `npm install` do?

Reads `package.json` and installs all listed dependencies into `node_modules`. Also generates or updates `package-lock.json`.

Q4. What is the difference between `npm install` and `npm ci`?

- `npm install` — installs and may update `package-lock.json`.
- `npm ci` — clean install strictly from `package-lock.json`. Used in CI/CD for reproducible builds.

Q5. What does `npm install --save-dev` mean?

Installs a package as a **devDependency** — only needed during development/testing, not production. Example: `@playwright/test`.

Q6. What is `npx` and how does it differ from `npm`?

- `npm` — installs packages.
- `npx` — executes packages without globally installing them. Example: `npx playwright test`.

Q7. How do you install Playwright browsers after installing the package?

```
npx playwright install
```

Downloads Chromium, Firefox, and WebKit binaries used by Playwright.

Q8. What does `npm init` do?

Initializes a new Node.js project by creating a `package.json` file with project name, version, description, and entry point.

Q9. How do you install a specific version of a package?

```
npm install @playwright/test@1.44.0
```

Q10. What is `package-lock.json` and why is it important?

Locks the exact versions of all installed packages including transitive dependencies — ensuring every machine installs the exact same dependency tree.

Q11. What is the difference between `npm install playwright` and `npm install @playwright/test`?

Command	What it installs	Use case
`npm install playwright --save-dev`	Base Playwright **library** only — browser automation API (`chromium.launch()`, `page.click()`) but **no test runner**	Use when building a custom automation tool, web scraper, or integrating with another test framework (Jest, Mocha)
`npm install @playwright/test --save-dev`	Playwright **test runner** — includes the library PLUS `test()`, `expect()`, fixtures, config, reporters, parallelism	Use for writing automated test suites with Playwright's own framework
`npm init playwright@latest`	Scaffolds a full project — installs `@playwright/test`, creates `playwright.config.ts`, example tests, and GitHub Actions workflow	Use when starting a new Playwright project from scratch

In most cases, use `@playwright/test` — it includes everything.

Q12. What is the correct syntax for `--save-dev` and what does the flag do?

The correct flag is `--save-dev` (one dash before dev) or the shorthand `-D`:

```
npm install @playwright/test --save-dev
npm install @playwright/test -D        # same thing
```

A common typo is `--save--dev` (double dash) — this does NOT work. `--save-dev` adds the package to `devDependencies` in `package.json` instead of `dependencies`.

01.2 — Packages

Topic group: Node.js & npm Ecosystem | **Sequence:** 2 of 37

Q1. What is a package in Node.js/npm?

A reusable module of code published to the npm registry. It has a `package.json` describing its name, version, and dependencies. Example: `@playwright/test`.

Q2. What is the difference between a dependency and a devDependency?

- `dependencies` — required at runtime (production). Example: `express`.
- `devDependencies` — required only during development/testing. Example: `@playwright/test`, `typescript`.

Q3. Where are dependencies listed in a project?

In `package.json`:

```
{
  "dependencies": { "express": "^4.18.0" },
  "devDependencies": { "@playwright/test": "^1.44.0" }
}
```

Q4. What does ^ mean in a version like `^1.44.0`?

Compatible with `1.44.0` — allows updates `>=1.44.0` and `<2.0.0` (same major). Minor and patch updates are automatic.

Q5. What does `~1.44.0` mean compared to `^1.44.0`?

- `~1.44.0` — patch updates only: `>=1.44.0` and `<1.45.0`.
- `^1.44.0` — minor + patch updates: `>=1.44.0` and `<2.0.0`.

Q6. What is a scoped package? Give an example.

A package belonging to an npm organization, prefixed with `@scope/`. Example: `@playwright/test` — `@playwright` is the scope, `test` is the package name.

Q7. What is `@playwright/test` specifically used for?

Playwright's **test runner package** — provides `test`, `expect`, fixtures, and the `playwright.config.ts` configuration system.

Q8. How do you see all installed top-level packages?

```
npm list --depth=0
```

Q9. How do you update a package to its latest version?

```
npm install @playwright/test@latest
```

Q10. How do you remove a package?

```
npm uninstall @playwright/test
```

Removes from both `node_modules` and `package.json`.

01.3 — node_modules

Topic group: Node.js & npm Ecosystem | **Sequence:** 3 of 37

Q1. What is the `node_modules` folder?

The directory where npm installs all packages and their transitive dependencies. Created automatically when you run `npm install`.

Q2. Should `node_modules` be committed to version control?

No. Add it to `.gitignore`. It can be hundreds of MB and can always be regenerated from `package.json` / `package-lock.json` via `npm install`.

Q3. How do you regenerate `node_modules` after deleting it?

```
npm install
```

Q4. Why can `node_modules` be very large?

npm installs not just your direct dependencies but all **transitive dependencies** (dependencies of dependencies). One package can pull in dozens of others.

Q5. What is inside `node_modules/@playwright/test`?

Compiled JavaScript source, TypeScript type declaration files (`.d.ts`), and the CLI binary. This is what runs when you execute `npx playwright test`.

Q6. Can two packages inside `node_modules` have different versions of the same dependency?

Yes. npm uses **nested `node_modules`** to resolve version conflicts — each package can have its own subfolder with the version it needs.

Q7. What is the difference between `node_modules` and the npm registry?

- **npm registry** (registry.npmjs.org) — remote server where packages are stored.
- **`node_modules`** — local folder where packages are downloaded for a specific project.

Q8. How do you check if a specific package exists in `node_modules`?

```
npm list @playwright/test
```

Q9. What happens if you run `npm install` and `node_modules` already exists?

npm checks `package-lock.json` and only installs missing or outdated packages. Correct packages are not re-downloaded.

Q10. Why should you not manually edit files inside `node_modules`?

Changes will be overwritten the next time `npm install` runs. Use `patch-package` or fork the package for permanent changes.

02.1 — import (TypeScript/ES Modules)

Topic group: TypeScript Essentials | **Sequence:** 4 of 37

Q1. What does `import` do in TypeScript?

Brings exported values (functions, constants, types) from another module/package into the current file.

Q2. What is the difference between default import and named import?

- **Named import** — uses {} to import specific exports: `import { test, expect } from '@playwright/test';`
- **Default import** — no {}, imports the single default export: `import defineConfig from '@playwright/test';`

Q3. What does `{ test, expect }` mean in the import statement?

Destructuring — extracts `test` and `expect` from the named exports of `@playwright/test`, making them available directly in the file.

Q4. What is `from '@playwright/test'`?

Specifies the **module path** to import from. Node.js resolves this to `node_modules/@playwright/test`.

Q5. Can you rename an imported value? How?

Yes, using `as`: `import { test as pwTest } from '@playwright/test';`

Q6. What is a namespace import (`import * as`)?

Imports all named exports into a single object:

```
import * as pw from '@playwright/test';
pw.test('my test', async ({ page }) => { ... });
```

Q7. What is the difference between `import` (ES Module) and `require` (CommonJS)?

Feature	`import` (ESM)	`require` (CJS)
Syntax	`import { x } from 'pkg'`	`const { x } = require('pkg')`
Loading	Static (compile-time)	Dynamic (runtime)
Tree-shaking	Supported	Not supported
TypeScript	Preferred	Legacy

Q8. Why use `import` over `require` in Playwright projects?

TypeScript's `import` is statically analysed — full IntelliSense, type checking, and auto-complete. `require` is a runtime function with no static type info.

Q9. What happens if you import a member that doesn't exist in the package?

TypeScript shows a **compile-time error**: `Module '"@playwright/test"' has no exported member 'xyz'`. Catches mistakes before running tests.

Q10. Where does TypeScript resolve the types for imported packages?

From `.d.ts` type declaration files in `node_modules`. For `@playwright/test`, types are in `node_modules/playwright/types/test.d.ts`.

02.2 — async

Topic group: TypeScript Essentials | **Sequence:** 5 of 37

Q1. What does `async` mean in TypeScript/JavaScript?

Marks a function as **asynchronous** — it always returns a `Promise` and allows the use of `await` inside it.

Q2. Why is the Playwright test callback marked as `async`?

All Playwright browser actions return Promises. `async` is required to use `await` on those Promises inside the test:

```
test('example', async ({ page }) => {
    await page.goto('https://example.com');
});
```

Q3. What does an `async` function return?

Always a `Promise`, even for plain values — `return 42` becomes `Promise.resolve(42)` automatically.

Q4. Can you use `await` inside a non-async function?

No. `await` outside an `async` function is a **syntax error**.

Q5. What is the difference between a regular and an async function?

Feature	Regular	Async
Return type	Any value	Always a `Promise`

Feature	Regular	Async
Can use `await`	No	Yes
Execution	Synchronous	Asynchronous (non-blocking)

Q6. What is an async arrow function? Give an example.

```
const login = async ({ page }) => {
    await page.goto('https://example.com/login');
    await page.fill('#username', 'admin');
};
```

Q7. What happens if an async function throws an error?

The returned `Promise` is **rejected** with that error. In Playwright tests, this fails the test.

Q8. Can you have multiple `await` calls in one async function?

Yes — each `await` pauses until that Promise resolves, then moves to the next line sequentially.

Q9. Does `async/await` make code run in parallel?

No. It makes async code **read sequentially** but does not run operations in parallel. Use `Promise.all()` for true parallelism.

Q10. Why is `async/await` preferred over `.then()` chaining in Playwright?

Produces cleaner, more readable code that looks synchronous. `.then()` chains become deeply nested and hard to debug with many sequential browser actions.

02.3 — await

Topic group: TypeScript Essentials | **Sequence:** 6 of 37

Q1. What does `await` do?

Pauses execution of an `async` function until the given `Promise` resolves, then resumes with the resolved value.

Q2. Why must every Playwright action use `await`?

All Playwright actions return Promises. Without `await`, the next line runs before the action completes — causing race conditions and flaky tests.

Q3. What is the difference between using `await` and not using it?

```
// WITHOUT await — wrong
page.goto('https://example.com'); // fires but doesn't wait
page.click('#button');            // runs before page loads

// WITH await — correct
await page.goto('https://example.com');
await page.click('#button');
```

Q4. What value does `await` return?

The resolved value of the Promise:

```
const title = await page.title(); // returns string "Google"
```

Q5. What happens if an awaited Promise is rejected?

An error is thrown at that line, stopping the function. In Playwright tests, this fails the test with the rejection message.

Q6. Can you `await` a non-Promise value?

Yes — `await 42` returns 42 immediately. Safe but unnecessary.

Q7. What is the difference between `await` and `.then()`?

Both handle resolved Promises, but `await` produces linear readable code vs deeply nested `.then()` chains.

Q8. How do you run multiple `await` operations in parallel?

```
await Promise.all([
    page.waitForNavigation(),
    page.click('#submit'),
]);
```

Q9. What is `await expect(...)` in Playwright?

Playwright assertions are asynchronous — they poll the DOM until conditions are met or time out. Must be awaited:

```
await expect(page).toHaveTitle('Google');
await expect(page.locator('#btn')).toBeVisible();
```

Q10. What error do you get if you forget `await` on a Playwright action?

TypeScript/ESLint warns: `Promise returned from ... is ignored`. At runtime: action fires without waiting, assertions run on incomplete state, tests become non-deterministic.

03.1 — Playwright Architecture & Browser Context

Topic group: Playwright Project Setup | **Sequence:** 7 of 37 **Ref guide:** BROWSER_CONTEXT_PAGE_GUIDE.md

---

Q1. What is the Playwright architecture hierarchy?

```
Browser
  └── BrowserContext (isolated session)
        └── Page (a browser tab)
```

- **Browser** — the browser process (Chromium, Firefox, WebKit).
- **BrowserContext** — isolated session (like incognito). Has own cookies, localStorage, auth state.
- **Page** — a single tab within a context. Each test gets a fresh page from its own context.

Q2. What is a BrowserContext and why is it important?

The key unit of test isolation — tests in different contexts share zero cookies, localStorage, or state. Multiple contexts can run in parallel in one browser:

```ts
const adminCtx = await browser.newContext({ storageState: 'auth/admin.json' });
const userCtx  = await browser.newContext({ storageState: 'auth/user.json' });
```

Q3. What is the difference between `browser.newPage()` and `browser.newContext().newPage()`?

- `browser.newPage()` — page in the **default context** (shared state).
- `browser.newContext().newPage()` — fully isolated context first, then page. Zero shared state. Playwright Test automatically gives each test its own BrowserContext.

Q4. What browsers does Playwright support?

Chromium, Firefox, and WebKit (Safari engine). All three from a single API. Configure via `projects` in `playwright.config.ts`.

Q5. What is headless mode?

- **Headless** — browser runs without visible UI (default). Fast, used in CI/CD.
- **Headed** — browser window is visible. Used for debugging.

```
npx playwright test --headed
```

Q6. What is the Playwright Test runner vs the Playwright library?

- **`playwright` library** — low-level API: `chromium.launch()`, `page.click()`. No test runner.
- **`@playwright/test`** — built on the library. Adds `test()`, `expect()`, fixtures, hooks, reporters, parallelism, and tracing.

Q7. What is `storageState` and how does it speed up tests?

Saves browser cookies and localStorage to JSON after login. All tests load it instead of re-logging in:

```
// Save after login (globalSetup)
await page.context().storageState({ path: 'auth/user.json' });

// Use in config
use: { storageState: 'auth/user.json' }
```

Q8. How does Playwright auto-waiting work?

Before any action (click, fill, hover), Playwright checks: element is attached, visible, stable (not animating), not obscured, and enabled. Retries up to the action timeout. Eliminates explicit waits.

Q9. What protocols does Playwright use to communicate with browsers?

- **Chromium** — Chrome DevTools Protocol (CDP)
- **Firefox** — Firefox Remote Protocol
- **WebKit** — WebKit Inspector Protocol All are direct WebSocket connections — much faster than Selenium's HTTP WebDriver.

Q10. What is `globalSetup` and `globalTeardown`?

Functions that run **once before/after all test files**:

```
// playwright.config.ts
globalSetup: './globalSetup.ts',
globalTeardown: './globalTeardown.ts'
```

Used for login (save storage state), DB seeding, or starting mock servers.

03.2 — playwright.config.ts

Topic group: Playwright Project Setup | **Sequence:** 8 of 37

Q1. What is `playwright.config.ts`?

Central configuration file controlling: test directory, timeouts, browsers, base URL, retries, reporters, parallelism, screenshots, videos, and more.

Q2. What function is used to define the config?

`defineConfig()` from `@playwright/test` — provides TypeScript type-checking and IntelliSense:

```
import { defineConfig } from '@playwright/test';
export default defineConfig({ ... });
```

Q3. What does `testDir` do?

Specifies where Playwright looks for test files: `testDir: './tests'`

Q4. What does `timeout` control?

Maximum time (ms) one test can run before failing: `timeout: 30 * 1000` (30 seconds).

Q5. What is the `use` block?

Default options applied to all tests — `baseURL`, `headless`, `screenshot`, `video`, `viewport`, `storageState`, etc.

```
use: {
  baseURL: 'https://example.com',
  screenshot: 'only-on-failure',
  video: 'retain-on-failure',
}
```

Q6. What are `projects` in the config?

Run same tests across multiple browsers or configurations:

```
projects: [
  { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
  { name: 'firefox',  use: { ...devices['Desktop Firefox'] } },
]
```

Q7. What does `retries` do?

Number of times a failed test is automatically retried: `retries: 2`

Q8. What is `fullyParallel`?

`fullyParallel: true` — all tests run in parallel, both across files AND within files.

Q9. What is `reporter` in the config?

Defines output format for results:

```
reporter: [['html'], ['junit', { outputFile: 'results.xml' }]]
```

Q10. Can you have multiple config files for different environments?

Yes: `npx playwright test --config=playwright.staging.config.ts`

03.3 — test() & Fixtures

Topic group: Playwright Project Setup | **Sequence:** 9 of 37

Q1. What is `test` in Playwright?

A named export from `@playwright/test` typed as `TestType<...>`. Callable as a function to register test cases, and has methods: `.describe`, `.beforeEach`, `.skip`, `.only`, etc.

Q2. What is the signature of a Playwright test?

```
test('test name', async ({ page }) => {
    // test steps
});
```

- First arg: test **name/title**
- Second arg: async **callback** with destructured fixtures

Q3. What are fixtures in Playwright?

Pre-built objects injected by Playwright before each test. Built-in fixtures: - page — fresh browser page - `context` — browser context - `browser` — browser instance - `request` — for API testing

---

Q4. How do you create a custom fixture?

```
export const test = base.extend<{ loginPage: LoginPage }>({
    loginPage: async ({ page }, use) => {
        await use(new LoginPage(page));
    }
});
```

Q5. What is `test.describe`?

Groups related tests under a named block:

```
test.describe('Login', () => {
    test('valid login', async ({ page }) => { ... });
    test('invalid password', async ({ page }) => { ... });
});
```

Q6. What is `test.only`?

Runs only that test, skipping all others in the file. For debugging.

Q7. What is `test.skip`?

Skips the test — it appears as “skipped” in the report.

Q8. What are `test.beforeEach` and `test.afterEach`?

Run before/after every test in the file or describe block:

```
test.beforeEach(async ({ page }) => { await page.goto('/'); });
```

Q9. Can a test override the global timeout?

Yes: `test.setTimeout(60 * 1000);` inside a test triples or sets a specific timeout.

Q10. What is the difference between test fixtures and worker fixtures?

Type	Scope	Created
Test fixture	Per test	Fresh for every test
Worker fixture	Per worker	Shared across tests in that worker

Use worker fixtures for expensive shared setup like a pre-authenticated browser session.

03.4 — Running Tests (CLI)

Topic group: Playwright Project Setup | **Sequence:** 10 of 37

Q1. How do you run all Playwright tests?

```
npx playwright test
```

Q2. How do you run a specific test file?

```
npx playwright test example.spec.ts
```

Q3. How do you run tests in headed mode?

```
npx playwright test --headed
```

Q4. How do you open the Playwright Inspector (debug mode)?

```
npx playwright test --debug
```

Q5. How do you open Playwright UI mode?

```
npx playwright test --ui
```

Q6. How do you run tests on a specific browser?

```
npx playwright test --project=chromium
npx playwright test --project=firefox
```

Q7. How do you run a test by its title (grep)?

```
npx playwright test -g "login test"
```

Q8. How do you open the HTML report after running?

```
npx playwright show-report
```

Q9. How do you override retries from the CLI?

```
npx playwright test --retries=2
```

Q10. What is the difference between running by filename vs `--grep`?

Command	Filters by
`npx playwright test example.spec.ts`	File name — all tests in that file
`npx playwright test -g "login"`	Test title — matching tests across all files

Q11. How do you correctly open Playwright UI mode?

Run without a file filter:

```
npx playwright test --ui
```

Then use the **left sidebar** to expand the file tree and find your tests, or use the **search bar** to filter by name.

Passing a file path with `--ui` (e.g., `npx playwright test example.spec.ts --ui`) opens UI mode but the file filter is **not applied as a CLI filter** — the UI opens with all tests and you must locate the file in the sidebar.

Q12. What are the key features of Playwright UI mode?

Feature	Description
Left sidebar	File tree — expand to browse all test files
Search bar	Filter tests by filename or test title
▶ Run button	Run a single test, a file, or all tests
Watch mode	Tests auto re-run on file save
Time-travel	Step through actions in the trace viewer inline
Step view	See each action with before/after DOM snapshots

Q13. Why do tests pass in normal mode but appear missing in UI mode?

Common causes: - **Passing a file filter with `--ui`** — UI mode ignores the CLI file filter; use the search bar inside the UI instead. - **Browser channel not found** — if `channel: 'chrome'` or `channel: 'msedge'` is configured but the browser isn't installed, the project silently fails. - **Tests not yet run** — in UI mode, tests start in an unrun state (grey). They appear in the sidebar but not in the results panel until you run them.

Diagnosis: Run without `--ui` first to confirm tests work:

```
npx playwright test example.spec.ts --reporter=list
```

Q14. How do you use system-installed browsers (Chrome/Edge) instead of Playwright's downloaded browsers?

Configure `channel` in `playwright.config.ts` projects — no browser download needed:

```
projects: [
  { name: 'chrome', use: { channel: 'chrome' } },   // system Google Chrome
  { name: 'msedge', use: { channel: 'msedge' } },   // system Microsoft Edge
]
```

Useful when corporate firewalls block `playwright install`.

Q15. What does `workers` default to when not explicitly set?

Playwright uses **half the logical CPU cores** of the machine:

`workers: undefined → Math.ceil(logicalCores / 2)`

For example: 8-core CPU → 4 workers. Control it explicitly:

```
workers: process.env.CI ? 1 : 2,   // conservative
workers: '50%',                      // same as default
workers: 1,                          // serial (easy for debugging)
```

03.5 — Storage State, Auth & Environment

Topic group: Playwright Project Setup | **Sequence:** 11 of 37 **Covers:** Storage State, Global Setup, Environment Variables, test.use(), multi-role testing, tsconfig

---

Q1. What is Storage State in Playwright?

Saves browser cookies and localStorage to a JSON file after logging in. Tests load it to start already authenticated — skipping login every time.

```
await page.context().storageState({ path: 'auth/user.json' });
```

Q2. How do you configure Storage State in the config?

```
globalSetup: './globalSetup.ts',
use: { storageState: 'auth/user.json' }
```

`globalSetup.ts` logs in once and saves the state. Every test starts authenticated.

Q3. How do you use environment variables in Playwright?

```
// .env file
BASE_URL=https://staging.example.com

// playwright.config.ts
import dotenv from 'dotenv';
```

```
dotenv.config();
use: { baseURL: process.env.BASE_URL }
```

Q4. How do you pass env variables from the CLI?

```
BASE_URL=https://prod.example.com npx playwright test
```

Q5. How do you use `test.use()` to override config for a specific describe block?

```
test.describe('Mobile', () => {
    test.use({ viewport: { width: 390, height: 844 } });
    test('mobile menu', async ({ page }) => { ... });
});
```

Q6. How do you test multiple user roles using storage states?

```
test.describe('Admin', () => {
    test.use({ storageState: 'auth/admin.json' });
    test('admin panel access', async ({ page }) => { ... });
});
test.describe('User', () => {
    test.use({ storageState: 'auth/user.json' });
    test('no admin access', async ({ page }) => { ... });
});
```

Q7. What is `tsconfig.json` in a Playwright project?

TypeScript compiler config. Used for IDE type checking, path aliases, and strict mode. Playwright compiles `.ts` internally but `tsconfig.json` is needed for IDE support.

Q8. How do you set path aliases in `tsconfig.json`?

```
{ "compilerOptions": { "paths": { "@pages/*": ["pages/*"] } } }
```

Then: `import { LoginPage } from '@pages/LoginPage';`

Q9. What is `test.step()`?

Groups actions into named steps visible in HTML report and trace viewer:

```
await test.step('Login', async () => {
    await page.goto('/login');
    await page.getByRole('button', { name: 'Login' }).click();
});
```

Q10. What is the difference between globalSetup, beforeAll, and beforeEach?

Hook	Runs	Scope
`globalSetup`	Once before ALL test files	Entire suite
`beforeAll`	Once before all tests in a file/describe	File/block

Hook	Runs	Scope
`beforeEach`	Before every individual test	Each test

03.6 — Playwright vs Selenium vs Cypress

Topic group: Playwright Project Setup | **Sequence:** 12 of 37

Q1. What are the key advantages of Playwright over Selenium?

Feature	Playwright	Selenium
Auto-waiting	Built-in	Manual explicit waits
Speed	Faster (CDP/WebSocket)	Slower (HTTP WebDriver)
Multi-browser	Chromium, Firefox, WebKit	Chromium, Firefox, Edge
Multi-tab	Native	Complex/fragile
Network mocking	Built-in `route()`	External proxy tools
Setup	Single `npm install`	Browser drivers per browser

Q2. What are the key differences between Playwright and Cypress?

Feature	Playwright	Cypress
Languages	JS, TS, Python, Java, C#	JS, TS only
WebKit/Safari	Yes	No
Multi-tab	Yes	No
iFrames	Easy (`frameLocator`)	Limited
Parallelism	Free (workers)	Paid (Cypress Cloud)
Architecture	Out-of-process	In-process (runs in browser)

Q3. When would you choose Playwright over Selenium?

When you need fast reliable tests without explicit waits, WebKit/Safari coverage, multi-tab automation, network interception, modern TypeScript tooling, or built-in tracing and screenshots.

Q4. What are Playwright's key standout features?

Auto-waiting, cross-browser (including WebKit), multi-language support, network mocking, storage state, parallel workers, built-in API testing, visual comparison, trace viewer, and MCP/AI integration.

Q5. What is Playwright's architecture advantage over Selenium?

Playwright uses direct browser protocols (CDP, Firefox Remote Protocol, WebKit Inspector) over WebSocket. Selenium uses HTTP-based WebDriver — slower and more error-prone.

Q6. How does Playwright compare to Cypress for iFrame handling?

Playwright: `page.frameLocator('iframe#id').getByRole('button').click()` — simple chain. Cypress: No direct iFrame support — requires third-party plugins or `cy.iframe()`.

Q7. Can Playwright test mobile viewports?

Yes — use device emulation from Playwright's device registry:

```
use: { ...devices['iPhone 14'] }
```

Q8. Does Playwright support API testing natively?

Yes — via the built-in `request` fixture. No separate tool needed for REST API testing within test suites.

Q9. How does Playwright handle flaky tests compared to Selenium?

Playwright's auto-waiting and auto-retrying assertions eliminate most causes of flakiness. `retries` config and trace-on-retry provide safety nets. Selenium requires manual waits which are a primary source of flakiness.

Q10. What is Playwright MCP?

Model Context Protocol server that allows AI assistants (like GitHub Copilot) to control a live browser — inspect elements, navigate, and generate Playwright test code via natural language.

03.7 — Emulation (Mobile, Geolocation, Locale)

Topic group: Playwright Project Setup | **Sequence:** 13 of 37 **Ref:** playwright.dev/docs/emulation

---

Q1. What is emulation in Playwright?

Emulation simulates different devices, locales, timezones, geolocation, and permissions in a browser context — without needing a real device.

```
import { devices } from '@playwright/test';
use: { ...devices['iPhone 14'] }
```

Q2. How do you emulate a mobile device in Playwright?

Use device descriptors from Playwright's built-in device registry:

```
// playwright.config.ts
projects: [
    { name: 'Mobile Chrome', use: { ...devices['Pixel 7'] } },
    { name: 'Mobile Safari', use: { ...devices['iPhone 14'] } },
]
```

Each device preset sets `viewport`, `userAgent`, `deviceScaleFactor`, `hasTouch`, and `isMobile` automatically.

Q3. How do you manually set viewport size?

```
use: { viewport: { width: 1280, height: 720 } }

// Or per-test
test.use({ viewport: { width: 390, height: 844 } });

// Or during a test
await page.setViewportSize({ width: 375, height: 667 });
```

Q4. How do you emulate geolocation?

```
// playwright.config.ts
use: {
    geolocation: { longitude: 12.492507, latitude: 41.889938 }, // Rome, Italy
    permissions: ['geolocation'],
}

// Override per test
await context.setGeolocation({ latitude: 37.7749, longitude: -122.4194 });
```

Q5. How do you emulate a specific locale and timezone?

```
use: {
    locale: 'fr-FR',
    timezoneId: 'Europe/Paris',
}
```

This affects `Intl` APIs, `Date.toLocaleString()`, and browser UI locale.

Q6. How do you emulate dark mode or color scheme?

```
use: { colorScheme: 'dark' }  // 'light' | 'dark' | 'no-preference'

// Override in test
await page.emulateMedia({ colorScheme: 'dark' });
```

Q7. How do you grant or deny browser permissions?

```
// Grant
await context.grantPermissions(['geolocation', 'notifications']);
```

```ts
// Clear all permissions
await context.clearPermissions();
```

Q8. How do you emulate a device with touch support?

Touch is automatically set when using device descriptors. For manual setup:

```ts
use: {
    hasTouch: true,
    isMobile: true,
}
```

Q9. How do you emulate network conditions (offline/slow)?

```ts
// Simulate offline
await context.setOffline(true);

// Or per page
await page.context().setOffline(true);
```

Q10. How do you get a list of available device presets?

```ts
import { devices } from '@playwright/test';
console.log(Object.keys(devices));
// ["iPhone 14", "Pixel 7", "iPad Pro 11", "Galaxy S8", ...]
```

Or browse the full list at: `playwright.dev/docs/emulation#devices`

03.8 — Browser, BrowserContext & Page (Raw API)

Topic group: Playwright Project Setup | **Sequence:** 8 of 37 **Ref guide:** BROWSER_CONTEXT_PAGE_GUIDE.md

---

Q1. What is the three-layer hierarchy in Playwright?

```
Browser
  └── BrowserContext  (isolated session — own cookies, localStorage, cache)
        └── Page      (a single browser tab)
```

Layer	Cost to create	Isolation
Browser	High (launches process)	—
BrowserContext	Low	Full (no shared state)
Page	Very low	Within context

Q2. What is the difference between the `playwright` package and `@playwright/test`?

- **`playwright`** — raw browser-automation library. You manually launch browsers, create contexts and pages.
- **`@playwright/test`** — built on top of `playwright`. Adds `test()`, `expect()`, fixtures, hooks, reporters, parallelism, and tracing. **Preferred for test suites.**

```
// playwright (raw)
import { chromium, firefox, webkit } from 'playwright';
const browser = await firefox.launch({ headless: false });

// @playwright/test (test runner)
import { test, expect } from '@playwright/test';
test('my test', async ({ page }) => { /* page is injected */ });
```

Q3. How do you launch a browser using the raw Playwright API?

```
import { chromium, firefox, webkit } from 'playwright';

const browser = await chromium.launch();                    // headless (default)
const browser = await firefox.launch({ headless: false });  // headed (visible)
const browser = await webkit.launch({ slowMo: 100 });       // slow down by 100ms
```

You are responsible for calling `await browser.close()` when done.

Q4. What are the key Browser methods?

```
const browser = await chromium.launch();

await browser.newContext()          // create isolated session
await browser.newPage()             // page in default context (shared state)
browser.contexts()                  // array of all open contexts
browser.isConnected()               // true/false
await browser.version()             // e.g. "115.0.5790.98"
await browser.close()               // close all contexts and browser process
```

Q5. How do you create a BrowserContext and why should you use it?

A BrowserContext is an isolated session — its own cookies, localStorage, cache. Creating one prevents tests from sharing login state or cached data.

```
const context = await browser.newContext();

// With options
const context = await browser.newContext({
  viewport:    { width: 1280, height: 720 },
  locale:      'en-US',
  timezoneId:  'America/New_York',
  colorScheme: 'dark',
  headless:    false,
  ignoreHTTPSErrors: true,
  storageState: 'auth/user.json',   // load saved cookies/localStorage
});
```

Q6. What are the key BrowserContext methods?

```
await context.newPage()                    // create a new tab in this context
context.pages()                            // array of all open pages
await context.addCookies([...])            // add cookies
await context.cookies()                    // read cookies
```

```
await context.clearCookies()                // clear cookies
await context.storageState({ path: 'f' })  // save state to file
await context.grantPermissions(['geolocation'])
await context.setGeolocation({ latitude: 37.77, longitude: -122.4 })
await context.setOffline(true)
await context.close()                       // closes all pages in context
```

Q7. What are the key Page methods?

```
await page.goto('https://example.com')       // navigate
await page.title()                           // get page title (string)
await page.url()                             // current URL
await page.content()                         // full HTML
await page.reload()
await page.goBack()
await page.goForward()
await page.screenshot({ path: 'shot.png' })
await page.close()
page.context()                               // returns the owning BrowserContext
```

Q8. What is the difference between `browser.newPage()` and `context.newPage()`?

- `browser.newPage()` — creates a page inside the **default (shared) context**. State is shared with other pages created the same way.
- `context.newPage()` — creates a page in an **explicitly isolated context**. Recommended for clean test isolation.

```
// Shared state risk
const page = await browser.newPage();

// Proper isolation
const context = await browser.newContext();
const page    = await context.newPage();
```

Q9. How does a typical end-to-end login test look using the raw Playwright API?

```
import { test, expect, Browser, Page, Locator } from '@playwright/test';
import { firefox } from 'playwright';

test('Login test', async () => {
  const browser: Browser = await firefox.launch({ headless: false });
  const page: Page       = await browser.newPage();

  await page.goto('https://example.com/login');

  const emailId:     Locator = page.locator('#input-email');
  const password:    Locator = page.locator('#input-password');
  const loginButton: Locator = page.locator('input[value="Login"]');

  await emailId.fill('user@example.com');
  await password.fill('secret123');
  await loginButton.click();
```

```
  await page.screenshot({ path: 'login_success.png' });

  const pageTitle: string = await page.title();
  expect(pageTitle).toBe('My Account');

  await browser.close();
});
```

Note: `test` and `expect` come from `@playwright/test`; the browser is launched manually via `playwright`.

Q10. Why is `@playwright/test` preferred over the raw `playwright` API for test suites?

Concern	Raw `playwright`	`@playwright/test`
Browser/context lifecycle	Manual	Auto (fixtures)
Isolation per test	Manual	Auto
Parallelism	Manual	Built-in
Retries, reporters	Not included	Built-in
`expect` with auto-retry	No	Yes
Hooks (`beforeEach` etc.)	No	Yes

Use raw `playwright` only for scripts, utilities, or custom frameworks.

Q11. How do you properly close resources when using the raw API?

Always close in reverse order of creation — Page → Context → Browser:

```
const browser = await chromium.launch();
const context = await browser.newContext();
const page    = await context.newPage();

try {
  // test work
} finally {
  await page.close();      // optional — context.close() closes all pages
  await context.close();
  await browser.close();
}
```

Failing to call `browser.close()` leaves orphaned browser processes.

Q12. What is the `channel` option in `chromium.launch()` and when do you use it?

By default, `chromium.launch()` uses Playwright's **bundled Chromium** (downloaded during `npx playwright install`). Adding `channel` switches to a **browser already installed on your machine**:

```
// Uses Playwright's bundled Chromium (default)
const browser = await chromium.launch({ headless: false });

// Uses system-installed Google Chrome
const browser = await chromium.launch({ headless: false, channel: 'chrome' });
```

```
// Uses system-installed Microsoft Edge
const browser = await chromium.launch({ headless: false, channel: 'msedge' });
```

	Bundled Chromium	`channel: 'chrome'`
Source	Downloaded by Playwright	Your installed Chrome
Version	Pinned to Playwright version	Your current Chrome version
Use case	CI/CD, reproducible runs	Testing real Chrome behavior

Important: When using `channel` in the config `projects`, those channel values are also injected into raw `chromium.launch()` calls inside tests. Running under an `msedge` project overrides any hardcoded `channel: 'chrome'` in your test.

Q13. What browser does Playwright use by default, and does it run in incognito mode?

Default browser: When you call `chromium.launch()` without `channel`, Playwright uses its own **downloaded Chromium** — NOT your system Chrome or Edge.

```
// Uses Playwright's Chromium (bundled) — NOT system Chrome
const browser = await chromium.launch({ headless: false });

// Uses system-installed Chrome
const browser = await chromium.launch({ headless: false, channel: 'chrome' });
```

Default mode — incognito-like: Every `browser.newContext()` creates a fresh isolated session with no cookies, no history, no saved passwords — behaves like an incognito window.

Behaviour	Default (no channel)	With `channel: 'chrome'` + `newContext()`
Browser used	Playwright's Chromium	System Chrome
Cookies/history	None (incognito-like)	None (incognito-like)
Extensions	Not loaded	Not loaded (unless configured)

To run two users simultaneously with full isolation, create two separate BrowserContexts:

```
const browser = await chromium.launch({ headless: false, channel: 'chrome' });

const context1 = await browser.newContext(); // user 1 — isolated session
const page1    = await context1.newPage();

const context2 = await browser.newContext(); // user 2 — isolated session
const page2    = await context2.newPage();

// page1 and page2 share zero cookies/localStorage — completely independent
await context1.close();
await context2.close();
await browser.close();
```

Q14. What is `launchPersistentContext()` and how is it different from `launch()`?

	`chromium.launch()`	`chromium.launchPersistentContext()`
Returns	`Browser`	`BrowserContext` (directly)
Mode	Incognito-like (no profile)	Normal mode (with user profile)
Extensions	Not loaded	Loaded
Saved cookies/history	No	Yes (if real profile path given)

```
// launch() — incognito-like, returns Browser
const browser  = await chromium.launch({ headless: false });
const context  = await browser.newContext();
const page     = await context.newPage();

// launchPersistentContext() — normal mode, returns BrowserContext directly
// 1st arg: user profile directory path. '' = temporary directory
const browser  = await chromium.launchPersistentContext('', { headless: false });
const page     = await browser.newPage(); // creates an extra new tab
```

Use `launchPersistentContext()` when you need browser extensions, a real user profile, or non-incognito behavior.

Q15. How many pages does `launchPersistentContext()` open by default, and what are the two ways to get a page from it?

`launchPersistentContext()` **automatically opens one default page** (`pages()[0]`) when the browser starts.

Way 1 — `browser.newPage()` → creates an additional blank tab (2 pages now open):

```
const browser = await chromium.launchPersistentContext('', { headless: false });
const page    = await browser.newPage(); // new tab — pages()[0] still exists
```

Way 2 — `browser.pages()[0]` → reuses the default page (only 1 page open):

```
const browser = await chromium.launchPersistentContext('', { headless: false });
const pages   = browser.pages();
console.log(pages.length); // 1 — the default page
const page    = pages[0];  // reuse it — no extra tab opened
```

Prefer **Way 2** to avoid an unnecessary extra tab.

Q16. What is the `userDataDir` argument in `launchPersistentContext()` and what is the difference between `''` and a real path?

`launchPersistentContext(userDataDir, options)` — the first argument controls where Chrome stores the user profile.

Value	Behaviour
`''` (empty string)	Temporary directory — profile is **discarded** when browser closes. Fresh session every run.
`'./session'` (a path)	Persistent directory — profile is **saved** to disk. Cookies, login state, and history survive across runs.

```
// Temporary — no data saved after close
await chromium.launchPersistentContext('', { headless: false });

// Persistent — './session' folder is created and reused across runs
// On 2nd run: browser remembers cookies, stays logged in
await chromium.launchPersistentContext('./session', { headless: false, channel: 'chrom
```

Use a real path when you want to **skip login on repeated runs** by reusing saved session data.